Bitirme Projesi · Proje Özeti

Blockchain Tabanlı Öğrenci Temsilcisi Seçim Sistemi

Projenin ana mantığı: problem, blockchain'in rolü, mimari, bir oyun yolculuğu, güvenlik ve doğrulama

Hazırlayan: Muhammed Taha Can

Teknolojiler: Node.js, Express, PostgreSQL, SHA-256 / HMAC, JWT, bcrypt, HTML + Tailwind CSS + JavaScript

Tarih: Ekim 2026

1. Projenin Amacı

Bu proje, öğrencilerin öğrenci numarası ve kendilerine verilen tek kullanımlık bir oy koduyla oy kullandığı bir elektronik seçim sistemidir. Her oy, bir öncekine kriptografik olarak bağlanan bir **blok** olarak kaydedilir. Böylece kayıtlar sonradan değiştirilirse sistem bunu tespit eder.

Çözülen problem

Klasik bir elektronik seçim sisteminde oylar veritabanı tablosunda düz satırlar olarak durur. Veritabanına erişimi olan biri tek bir `UPDATE` komutuyla oyu değiştirebilir ve **bu değişiklik iz bırakmaz**. Seçmenin de oyunun gerçekten sayıldığını kontrol etme imkânı yoktur.

Projenin hedefleri

Hedef	Projede nasıl sağlandı
Değişikliğin tespit edilmesi	Oylar SHA-256 hash zinciri olarak saklanır; tek bir oyun değişmesi zinciri kırar.
Seçmen gizliliği	Zincirde öğrenci numarası yer almaz, gizli anahtarlı hash'i (HMAC) yer alır.
Yetkisiz oy kullanımının engellenmesi	Oy için öğrenci numarası + tek kullanımlık oy kodu gerekir.
Çift oyun engellenmesi	Her seçmen bir kez oy kullanabilir; kontrol transaction içinde ve kilitli yapılır.
Doğrulanabilirlik	Her seçmen aldığı makbuz numarasıyla (TxID) oyunun zincirde olduğunu kontrol edebilir.
Şeffaflık	Zincir herkese açıktır; dışa aktarılıp bağımsız olarak doğrulanabilir.

2. Blockchain'in Bu Projedeki Rolü

Hash nedir?

Hash fonksiyonu, herhangi bir veriyi sabit uzunlukta bir "parmak izine" çevirir. Projede **SHA-256** kullanılır ve çıktı her zaman 64 karakterdir. Üç önemli özelliği vardır:

- **Deterministik:** Aynı veri her zaman aynı hash'i üretir.
- **Çığ etkisi:** Verideki tek bir karakter değişse bile hash tamamen değişir.
- **Tek yönlü:** Hash'ten orijinal veriye geri dönülemez.

Blok yapısı

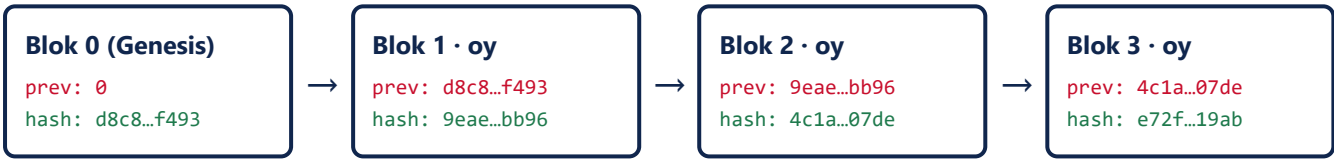
Sistemdeki her oy bir bloktur. Aşağıda veritabanından alınmış gerçek bir blok görülmektedir:

```
{
  "block_index":   1,
  "timestamp":     "2026-10-25T11:59:13.660Z",
  "data": {
    "voterHash":    "721db1c427fe9c1d...45db6771ac51f12fdd",   ← öğrenci numarası değil, HMAC'i
    "candidateId":  1,
    "provinceCode": 34,
    "txId":         "8503da475beeb55c...6d3b6b709aa2a54033cb63"  ← seçmenin makbuzu
  },
  "previous_hash": "d8c837627fcf7395...0b6683be997ba3bf493",   ← bir önceki bloğun hash'i
  "hash":          "9eaee19ee7d9de17...6f6c7d90321057bb96"     ← bu bloğun hash'i
}
```

Bloğun hash'i şu formülle hesaplanır:

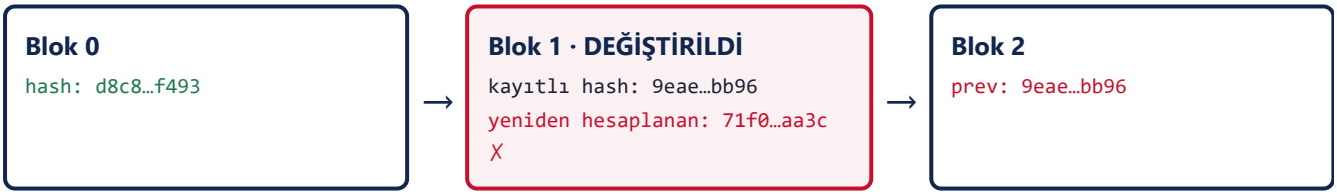
```
hash = SHA-256( block_index | timestamp | data | previous_hash )
```

`previous_hash` hash hesabına dahil olduğu için her blok kendinden öncekine **kilitlenir**. Zincirin ilk bloğu olan **Genesis blok**'ta önceki blok olmadığı için `previous_hash = "0"` değeri kullanılır.



Kurcalama neden yakalanır?

Biri veritabanında Blok 1'deki oyu değiştirirse iki şeyden biri olur:



1. Saldırgan hash'i güncellemezse, yeniden hesaplanan hash kayıtlı hash'le tutmaz: **"Blok 1: hash uyumsuzluğu"**.
2. Saldırgan Blok 1'in hash'ini de yeniden hesaplarsa, bu sefer Blok 2'nin içindeki `previous_hash` tutmaz: **"Blok 2: previousHash uyumsuzluğu"**.

Fark edilmeden değişiklik yapmak için değiştirilen bloktan sonraki bütün blokların yeniden hesaplanması gerekir. Sistem bunu da ek kontrollerle zorlaştırır (bkz. Bölüm 6).

3. Sistem Mimarisi

Proje **katmanlı mimari** ile geliştirilmiştir. Her katmanın tek bir sorumluluğu vardır ve her katman sadece bir altındaki katmanla konuşur.

Sunum Katmanı — Frontend (HTML + Tailwind CSS + JavaScript)
index.html (oy verme, canlı sonuçlar, zincir gezgini) · admin.html (yönetici paneli) · js/api.js (backend ile konuşan tek dosya) · js/i18n.js (Türkçe / İngilizce)

▼ HTTP + JSON (REST API) ▲

Uygulama Katmanı — Backend (Node.js + Express)
routes/ sadece HTTP: isteği al, servisi çağır, JSON dön · **middleware/** JWT + rol kontrolü, hız sınırlama, merkezi hata yönetimi · **services/** iş kuralları: oy, zincir, seçmen, sonuç · **utils/** saf kriptografi fonksiyonları

▼ Parametreli SQL ▲

Veri Katmanı — PostgreSQL
blocks (zincir) · allowed_voters (seçmen listesi + oy kodu hash'i) · candidates (adaylar) · users (yöneticiler)

Teknoloji seçimleri ve gerekçeleri

Teknoloji	Neden seçildi
Node.js + Express	Hafif ve hızlı bir REST API altyapısı sunar. Dahili `crypto` modülü SHA-256 ve HMAC desteği sağlar.
PostgreSQL	Transaction ve kilit (lock) desteği sayesinde eşzamanlı oylarda tutarlılık sağlanır. Blok verisi JSONB olarak esnek biçimde saklanır.
HTML + Tailwind + JS	Projenin ağırlığı backend'deki blockchain mantığındadır. Arayüz sade tutulmuş ve tüm hesaplamalar güvenli tarafta, sunucuda yapılmıştır.
JWT + bcrypt	Yönetici oturumu için endüstri standardı kimlik doğrulama ve yavaş, tuzlu şifre hash'leme.
Docker	Proje tek komutla, her bilgisayarda aynı şekilde çalıştırılabilir.

Kullanılan tasarım desenleri

Desen	Projedeki yeri
Katmanlı mimari	routes → services → db
Middleware zinciri (Chain of Responsibility)	İstek; kimlik doğrulama → rol kontrolü → route sırasıyla geçer.
Factory	`requireRole('admin')` ve hız sınırlayıcılar parametreye göre middleware üretir.
Singleton	Veritabanı bağlantı havuzu ve ayar nesnesi uygulama genelinde tektir.
Facade	Frontend'de `api.js`; backend'de `withTransaction()` karmaşık işlemleri tek arayüzün arkasında toplar.
Unit of Work	Oy kaydındaki tüm sorgular tek transaction içinde ya birlikte kalıcı olur ya da hiç olmaz.
Observer	Dil değişince abone modüller haberdar edilir ve metinlerini yeniden çizer.

4. Bir Oyun Yolculuğu

Seçmen "Oy Ver" butonuna bastığında tarayıcı `POST /api/vote` isteği gönderir. İstek sunucuda şu adımlardan geçer:

1. **Hız sınırlama:** Aynı IP adresinden kısa sürede çok fazla deneme varsa istek reddedilir (deneme-yanılma saldırısına karşı).
2. **Girdi kontrolü:** Öğrenci numarası 9 haneli mi, oy kodu girilmiş mi, aday geçerli mi?
3. **Transaction başlar.** Bundan sonraki adımlar ya hep birlikte gerçekleşir ya da hiçbiri gerçekleşmez.
4. **Seçmen doğrulama:** Öğrenci listede mi ve oy kodunun HMAC'i kayıtlı değerle eşleşiyor mu? Numara yoksa da kod yanlışsa da *aynı* hata mesajı döner. Böylece listede hangi numaraların olduğu dışarı sızmaz.
5. **Çift oy kontrolü:** Seçmen daha önce oy kullandıysa işlem durdurulur. Seçmen satırı kilitlendiği (`FOR UPDATE`) için aynı anda gelen iki istek bu kontrolü atlatamaz.
6. **Zincir kilidi:** PostgreSQL advisory lock ile zincire yazma sıraya alınır. Aynı anda 50 oy gelse de her biri sırayla, doğru blok numarasıyla eklenir.
7. **Blok oluşturma:** Son bloğun hash'i alınır; voterHash (HMAC), aday, il kodu ve TxID ile yeni blok hazırlanır ve hash'i hesaplanıp kaydedilir.
8. **Seçmen işaretlenir:** `has_voted = TRUE`
9. **COMMIT** → Seçmene makbuz döner: `txId` , blok numarası ve blok hash'i.

> **Makbuz doğrulama:** Seçmen TxID'sini "Oyunu Doğrula" alanına girerek oyunun zincirde olduğunu görür. Makbuz, kime oy verildiğini bilerek göstermez. Böylece makbuz, oy satın almak isteyen birine "kanıt" olarak gösterilemez.

Yönetici akışı

1. Yönetici JWT ile giriş yapar. Şifre veritabanında bcrypt hash'i olarak durur.
2. Seçmen listesini Excel'den yükler: A = sıra, B = öğrenci numarası, C = il plaka kodu.
3. Sistem her öğrenci için rastgele bir oy kodu üretir (örn. `K7M4-P9QX`). Kodun kendisi veritabanına **yazılmaz**, sadece HMAC'i saklanır. Kodlar yöneticiye bir kez, dağıtım listesi olarak Excel dosyasıyla verilir.
4. Yönetici istatistikleri izleyebilir, seçimi sıfırlayabilir veya listeyi temizleyebilir.

5. Güvenlik ve Gizlilik

Tehdit	Önlem
Bir oyun sonradan değiştirilmesi	SHA-256 hash zinciri ve zincir doğrulaması
Başkasının yerine oy kullanma	Tek kullanımlık, rastgele üretilmiş oy kodu
Zincirden kimin kime oy verdiğinin bulunması	Öğrenci numarası yerine **HMAC-SHA256**. Numaralar 9 haneli olduğu için düz SHA-256 deneme-yanılmayla dakikalar içinde çözülebilirdi; HMAC'te sunucudaki gizli anahtar bilinmeden bu mümkün değildir.
Çift oy / eşzamanlı istekler	Transaction + satır kilidi + zincir kilidi
Oy kodu / şifre tahmini	Hız sınırlama; şifreler bcrypt ile; aynı hata mesajı
Yetkisiz yönetim işlemi	JWT kimlik doğrulama + rol tabanlı yetkilendirme
SQL enjeksiyonu	Tüm sorgular parametreli
İç bilgi sızıntısı	Merkezi hata yakalayıcı: beklenmeyen hatalarda kullanıcıya genel mesaj, detay sadece sunucu logunda

6. Zincir Doğrulama Algoritması

`GET /api/chain/verify` uç noktası zinciri baştan sona şu kurallarla kontrol eder:

1. Genesis bloğun numarası 0, `previous_hash` değeri "0" ve hash'i doğru mu?
2. Her blok için:
   - Blok numarası sıralı mı? (silinen ya da araya eklenen blok)
   - `previous_hash` bir önceki bloğun hash'ine eşit mi? (bağlantı)
   - Blok içeriğinden yeniden hesaplanan hash kayıtlı hash'le aynı mı? (içerik değişikliği)
   - Aynı voterHash ikinci kez görünüyor mu? (çift oy)
3. **Çapraz kontrol:** Zincirdeki oy sayısı, seçmen listesinde "oy kullandı" olarak işaretlenen kişi sayısına eşit mi?

> **Neden çapraz kontrol?** Zincirin *son* bloğu silinirse geriye kalan zincir kendi içinde hâlâ tutarlıdır, hash kontrolleri bunu yakalayamaz. Oy sayısı ile "oy kullandı" kayıtlarının karşılaştırılması bu saldırıyı da ortaya çıkarır.

Doğrulama sonucu ana sayfadaki **Blockchain Durumu** alanında 5 saniyede bir güncellenir ("Zincir doğrulandı" ya da "Zincir bozuk" ve sebebi). Zincirin tamamı JSON olarak dışa aktarılıp bağımsız olarak doğrulanabilir.

Veritabanı şeması

Tablo	Sütunlar	Not
blocks	`block_index, timestamp, data (JSONB), hash, previous_hash`	timestamp metin olarak saklanır; böylece hash hesaplanırken birebir aynı kalır.
allowed_voters	`student_number, province_code, code_hash, has_voted`	Oy kodu düz metin olarak saklanmaz.
candidates	`id, name, party, color`	Sonuçlar doğrudan zincirden sayılır.
users	`username, password_hash, role`	bcrypt ile saklanan yönetici hesapları

7. Testler

Projede **62 otomatik test** bulunur (`npm test`). Testler gerçek veritabanına değil, ayrı bir test veritabanına bağlanır.

Test grubu	Kanıtladığı
Kriptografi (birim)	SHA-256 standart test vektörü, deterministik hash, HMAC'in düz SHA-256 olmadığı, oy kodu formatı ve benzersizliği
Zincir doğrulama (birim)	Geçerli zincir kabul edilir; değiştirilen veri, yeniden hesaplanan hash, bozuk sıra, sahte genesis ve çift seçmen yakalanır
Oy akışı (entegrasyon)	Doğru kodla oy, yanlış / başkasının kodu, çift oy, geçersiz aday, herkese açık zincirde öğrenci numarası olmaması, sonuçlar, makbuz
Zincir bütünlüğü (entegrasyon)	Gerçek veritabanında oy değiştirme, ortadan ve sondan blok silme tespiti
Eşzamanlılık	Aynı anda gönderilen 50 oyun hiçbiri kaybolmaz; aynı seçmenin aynı anda 5 denemesinden sadece 1'i kabul edilir
Yönetim (entegrasyon)	Giriş, bcrypt saklama, 401/403 erişim kontrolü, sahte IP başlığıyla hız sınırlamasının atlatılamaması, kod üretimi, sıfırlama ve temizleme

8. Kapsam, Sınırlılıklar ve Gelecek Çalışmalar

> **Kapsam:** Zincir tek bir PostgreSQL veritabanında tutulur. Bu yapı, Bitcoin gibi dağıtık ve eşler arası (peer-to-peer) bir ağ değil, **kurcalanmaya karşı kanıt bırakan bir defterdir** (tamper-evident ledger). Sistem değişikliği *tespit eder*, ancak hem veritabanına hem de sunucunun gizli anahtarlarına tam erişimi olan birinin zinciri baştan yeniden oluşturmasını tek başına *engelleyemez*.

Gelecek çalışmalar

- **Zincir başı sabitleme (anchoring):** Son blok hash'inin periyodik olarak bağımsız bir yere (ör. halka açık bir blockchain ya da zaman damgası servisi) yazılması. Böylece zincirin baştan yeniden yazılması da tespit edilebilir.
- **Çok düğümlü yapı:** Zincirin birden fazla bağımsız sunucuda tutulup uzlaşma (konsensüs) ile doğrulanması.
- **Tam anonimlik:** Kör imza (blind signature) ya da sıfır bilgi ispatı (zero-knowledge proof) ile sunucu yöneticisinin bile seçmen ile oy arasında bağ kuramaması.
- **Gerçek zamanlı güncelleme:** Periyodik sorgulama yerine WebSocket.

Sonuç

Proje; katmanlı bir backend, transaction ve kilitlerle tutarlı oy kaydı, HMAC ile seçmen gizliliği, tek kullanımlık oy kodlarıyla yetkilendirme ve SHA-256 hash zinciriyle değişiklik tespiti sunan, 62 otomatik testle doğrulanmış uçtan uca bir elektronik seçim sistemidir. Zincir herkese açık olduğu için sistemin güvenilirliği yöneticiye değil, herkesin bağımsız olarak doğrulayabilmesine dayanır.